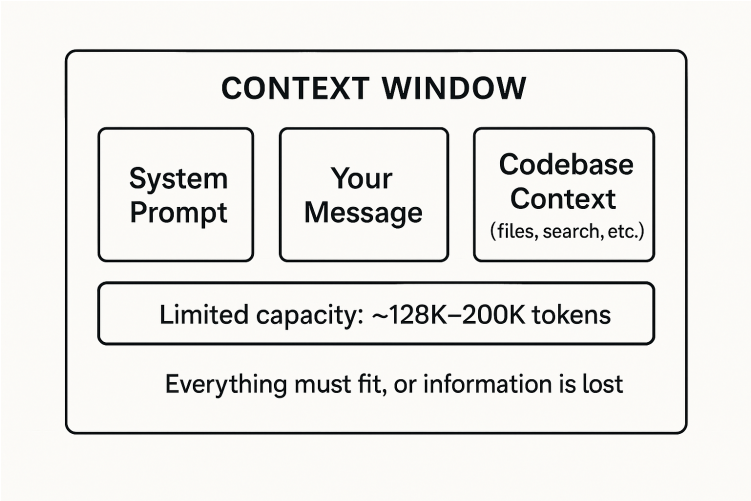
1. Preparing Code Repositories for AI tools

This section explains how to prepare your repository so that AI coding agents have the right context from the moment they start working — before they write a single line of code. The concepts here apply to **all AI coding tools** (GitHub Copilot, Amp, and others).

The Context Window and Discovery Tax

At the core of AI tools are Large Language Models (LLMs). When you interact with an AI agent, your prompt and relevant context are sent to an LLM which generates a response.

LLMs have a critical limitation: the **context window** - a fixed amount of text the model can "see" at once.



Without pre-defined instructions, every time an agent starts a task it must scan directories, read documentation, search for patterns, and discover frameworks. This happens before doing any real work.

This "discovery tax" wastes:
- **Time**. Multiple file reads and searches before real work begins
- **Tokens**. Consuming precious context window capacity
- **Cost**. More API calls, higher usage (depending on the subscription)
- **Focus**. The agent may miss important context or make wrong assumptions

Custom instructions eliminate the discovery tax by pre-loading the context an agent needs.

Instruction Hierarchy

Each tool has its own mechanism for loading instructions, but the core idea is the same: a **root instruction file** that is always loaded, plus **supplementary files** that provide deeper context for specific areas of the codebase.

Concept	GitHub Copilot	Amp
Root instructions	`copilot-instructions.md` in `.github/`	`AGENTS.md` in repo root
Project context files	`.github/instructions/*.instructions.md`	`.agents/instructions/*.md`
Agents / Skills	`.github/agents/*.agent.md`, `.github/skills/`	`.agents/skills/*/SKILL.md`
Prompts	`.github/prompts/*.prompt.md`	—

When multiple levels of instructions apply, they are **concatenated, not overridden**. Repository instructions complement organisation policies.

Tool-Specific Features

- **GitHub Copilot** combines instructions from multiple sources (organisation, repository, path-specific, and personal). Copilot Spaces can also provide context by pulling files, repositories, and notes from multiple sources into a shared, curated collection.
- **Amp** reads `AGENTS.md` at the repo root to discover available skills and project-level instructions. Skills are structured as `SKILL.md` files inside `.agents/skills/`.

Instruction Files

The ai-common boilerplate provides a structured set of instruction files:

Root Instruction File

The root file is always loaded by the agent. It defines behavioural guardrails: confidence scoring (0–10 scale, do not edit below 8), "investigate before assuming", and references to the instruction files below.

Tool	File	Location
Copilot	`copilot-instructions.md`	`.github/`
Amp	`AGENTS.md`	repo root

Project Context Templates

Multiple template files covering repository structure, development standards, project context, testing approach, CI/CD workflows, and usage patterns. Each ships with `<!-- TODO -->` placeholders ready to be populated:

Template	Content
`repo_overview`	Repo type, directory layout, package categories, key configuration files
`development_standards`	Naming conventions, frameworks and versions, build commands, code style rules
`project_context`	Business purpose, design principles, domain terminology and glossary
`testing_approach`	Test framework, file patterns, coverage expectations, mocking conventions
`workflows`	Branch strategy, PR process, conventional commits, versioning, CI/CD
`usage_patterns`	How consumers install, import, configure, and use the project

These templates can be populated automatically using the **Repository Scanner** workflow (see 2. Agentic Workflows, Agents & Skills - WIB Tech -- GenAI for Delivery - Westpac Enterprise Confluence), which analyses the codebase and fills in the placeholders.

Getting started

See the Onboarding Guide for easy setup instructions covering both Copilot and Amp, or the Composing Workflows guide for installer reference.

Contributing to ai-common

The ai-common repository is **tool-agnostic by design**. All agent, skill, and instruction content is authored once in `src/` and then transformed by a build step into tool-specific output.

```
src/             author content here (tool-agnostic source)
manifest.yaml    defines packages, file mappings, and metadata
      node tools/build.js
copilot/         generated output for GitHub Copilot (do not edit)
amp/             generated output for Amp (do not edit)
```

Do not edit files in `copilot/` or `amp/` directly — they are overwritten on every build. Always make changes in `src/` and run the build to regenerate.

For full details on adding agents, skills, and prompts, see the Contributing Guide.